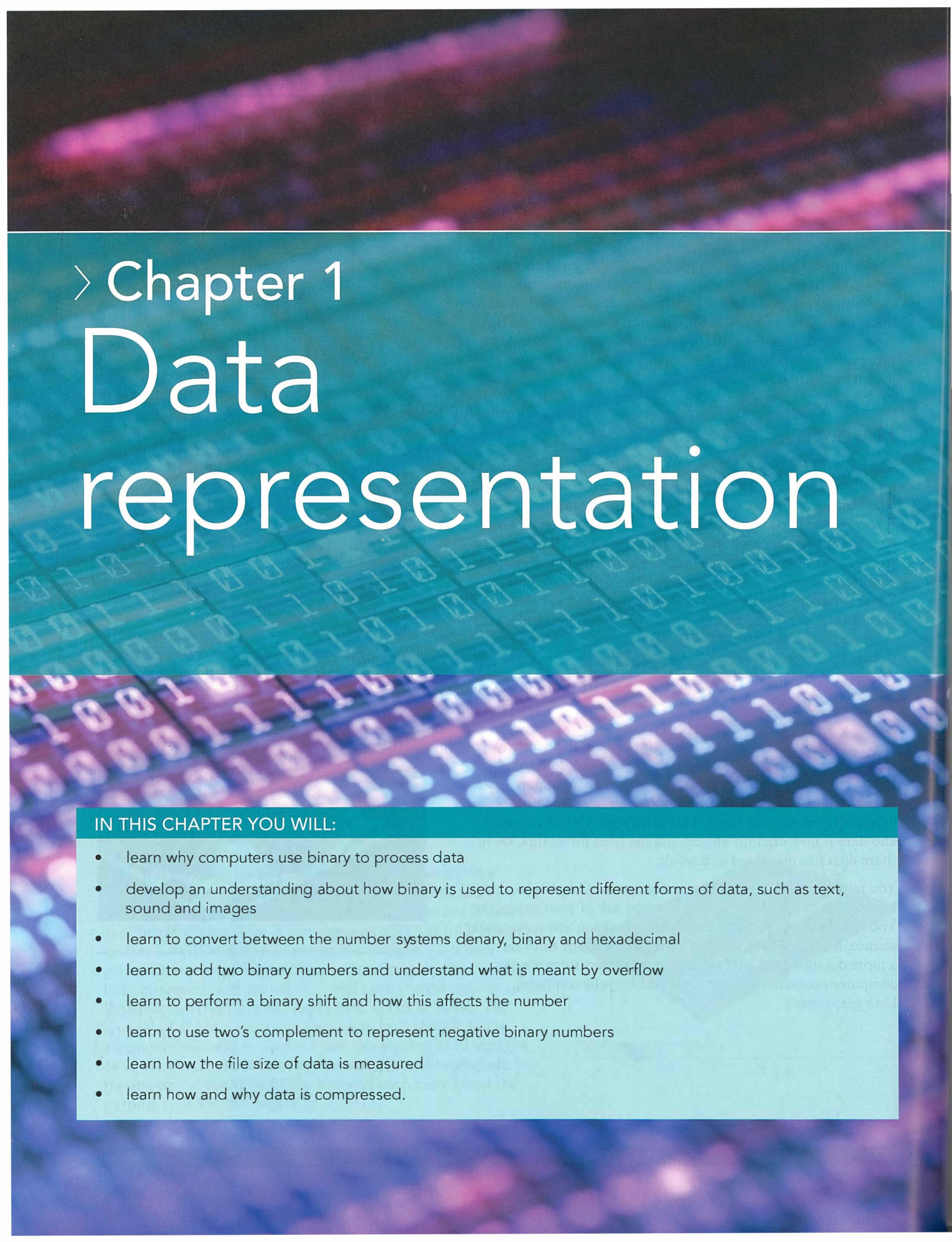


Chapter 1
Data representation

IN THIS CHAPTER YOU WILL:

- learn why computers use binary to process data
- develop an understanding about how binary is used to represent different forms of data, such as text, sound and images
- learn to convert between the number systems denary, binary and hexadecimal
- learn to add two binary numbers and understand what is meant by overflow
- learn to perform a binary shift and how this affects the number
- learn to use two's complement to represent negative binary numbers
- learn how the file size of data is measured
- learn how and why data is compressed.

GETTING STARTED

Working with a partner, write the words 'text', 'sound' and 'image' on a piece of paper. Around each word, write words that you can think of that are linked to each of the three words. For example, you could write the words 'sound wave' next to the word 'sound'.

DEBUGGING USING HEXADECIMAL

Computer programs are the foundation of computer games. If there is an error with the program, this can cause unexpected behaviour in the game. For example, a character may move the wrong way when a button is pressed. The programmer needs to find out why the character is moving incorrectly. The amount of code that needs to be assessed to find this error in a computer game is enormous and the binary values involved will be vast. Therefore, the programmer will often use **hexadecimal** to debug the error. One way this can be done is to look at the address for a variable. The address is the location in memory where the data is stored and it is shown in hexadecimal. This address is then looked up in a table to find the value held in the address. The value is also given as hexadecimal. If the value in the address is, for example, 0000 or FFFF (this can mean that the variable has not been initiated) the programmer will know that there is a problem with this value and has located the variable with which the error is occurring. The programmer can then find out why the variable is not holding a valid value.

If the addresses and values were given as **binary**, rather than as hexadecimal, then it would be more difficult to identify if an error has occurred. For example, it is easier to see if an address contains a valid value, e.g. F2A5, rather than 1111001010100101.

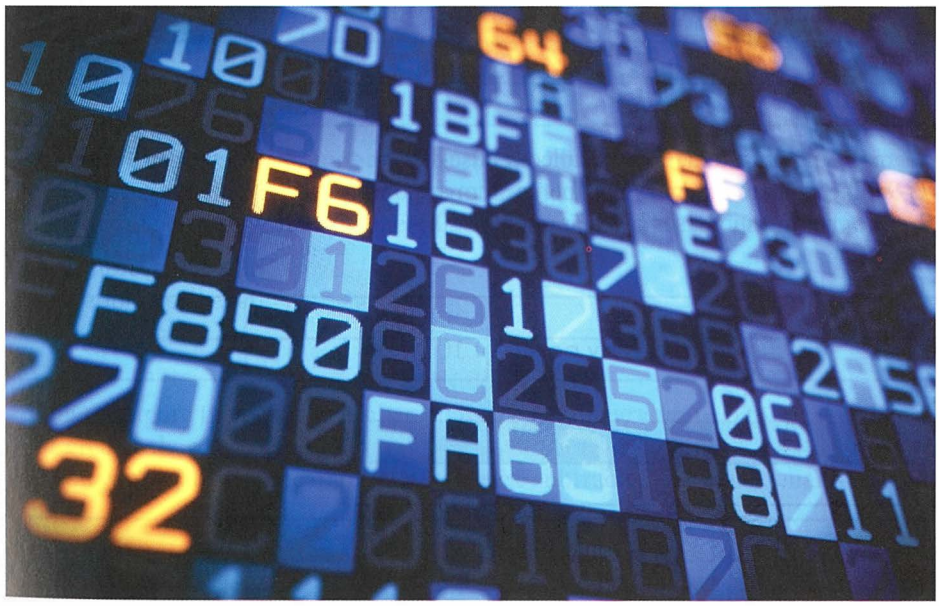


Figure 1.1: Data represented by hexadecimal letters and numbers

KEY WORDS

hexadecimal: a base-16 number system that uses the values 0–9 and characters A–F.

binary: a base-2 number system that uses the values 0 and 1.

CONTINUED

Discussion questions

1 Do you think that you have ever seen or used hexadecimal before? If you have, can you provide an example of where you have seen or used it?

2 Why is it important for a programmer to be able to effectively debug a computer program?

1.1 Binary and hexadecimal

Humans process a large amount of data on a daily basis. Did you know that when we have a conversation with someone, we process approximately 60 bits of information per second? Our brain is designed to process a maximum of about 120 bits per second. That's why it is so difficult to listen to two people talk to us at once!

As humans, we process data that is called **analogue data**. Everything we see, hear, feel and smell is a continuous stream of analogue data to our senses.

Questions

1 Can you think of **three** examples of analogue data?

2 Why is analogue data described as continuous data?

The components that process data in a computer are made up of very small **logic gates**. These gates cannot process analogue data, they can only process **digital data**. This data is stored in **registers**. You can find out more about logic gates in Chapter 10. Digital data is discrete data that only has two values, 1 and 0. Any data that we want a computer to process must be converted into digital data. This type of data is also called binary data.

KEY WORDS

analogue data: a continuous stream of data that is processed by humans.

logic gate: a very small component in a computer system that controls the flow of electricity.

digital data: discrete data that only uses the values 0 and 1.

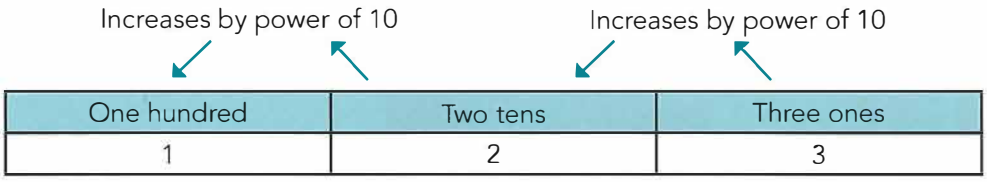
register: a small component in a computer system that is used to temporarily store data.

denary: a base-10 number system that uses the values 0–9.

Number systems

In our daily lives we mostly use a **denary** number system. This is called a base-10 number system and uses the digits 0–9. The units in a denary number system are ones, tens, hundreds and thousands. Each time the unit increases by the power of 10.

For example, the denary number 123 has:



One hundred	Two tens	Three ones
1	2	3

Computers use a binary number system. This is called a base-2 number system and uses the digits 0 and 1. The units in a binary number system increase by the power of 2.

Any denary values need to be converted to binary values so that they can be processed by a computer.

SKILLS FOCUS 1.1

CONVERTING BETWEEN DENARY AND BINARY NUMBERS

The size of a binary number is referred to by the number of bits it has. A 4-bit binary number will have 4 values that are either 1 or 0, for example, 1001. An 8-bit binary number will have 8 values that are also either 1 or 0. Binary values can get very large, some have as many as 64 bits; now that's a very big number! You will need to be able to understand binary numbers that have as many as 16 bits.

First, focus on converting denary numbers to 4-bit binary. To convert denary numbers to 4-bit binary the units that you need to use are 1, 2, 4 and 8. You can indicate with a 1 or a 0 whether that unit is required. For example, if you convert the denary number 13 to a 4-bit binary number you can use the following steps:

Write down the binary units from 8 to 1. An easy way to do this is using a table.

8	4	2	1

Do you notice how the units increase by the power of 2 each time (going from right to left)?

Compare the denary number (13) to the first binary unit (8) to see whether it is equal to or greater than the denary number. 13 is greater than 8, which means that the unit 8 is required and you can write a 1 beneath it. 8 can then be subtracted from the denary value, $13-8=5$.

8	4	2	1
1			

Now compare the denary number (5) to the next binary unit (4) to see whether it is equal to or greater than the denary number. 5 is greater than 4, which means that the unit 4 is required and you can write a 1 beneath it. 4 can then be subtracted from the denary value, $5-4=1$.

8	4	2	1
1	1		

Then compare the denary number (1) to the next binary unit (2) to see whether it is equal to or greater than the denary number. 1 is not greater than 2, which means that the unit 2 is not required and you can write a 0 beneath it. No subtraction is needed this time as you haven't used the unit.

8	4	2	1
1	1	0	

Finally, compare the denary number (1) to the last binary unit (1) to see whether it is equal to or greater than the denary number. 1 is equal to 1, which means that the unit is required and you can write a 1 beneath it. 1 can then be subtracted from the denary value. $1-1=0$.

8	4	2	1
1	1	0	1

CONTINUED

This means that if you convert the denary number 13 to 4-bit binary, you get the binary number 1101.

Questions

1 What is the largest denary number that can be created with a 4-bit binary number?

2 Convert the denary number 6 to a 4-bit binary number.

3 Convert the binary number 9 to a 4-bit binary number.

To convert denary numbers to 8-bit binary the units that you need to use are 1, 2, 4, 8, 16, 32, 64 and 128. You can indicate, again, with a 1 or a 0 whether that unit is required. For example, if you convert the denary number 150 to an 8-bit binary number you can use the same method as before:

Write down the binary units from 128 to 1:

128	64	32	16	8	4	2	1

Compare the denary number (150) to the first binary unit (128) to see whether it is equal to or greater than the denary number. 150 is greater than 128, which means that the unit 128 is required and you can write a 1 beneath it. 128 can then be subtracted from the denary value, 128 – 150 = 22.

128	64	32	16	8	4	2	1
1							

Next, compare the denary number (22) to the next binary unit (64) to see whether it is equal to or greater than the denary number. 22 is not greater than 64, which means that the unit 64 is not required and you can write a 0 beneath it. No subtraction is needed this time as you haven't used the unit.

128	64	32	16	8	4	2	1
1	0						

Then compare the denary number (22) to the next binary unit (32) to see whether it is equal to or greater than the denary number. 22 is not greater than 32, which means that the unit 32 is not required and you can write a 0 beneath it. No subtraction is needed this time as you haven't used the unit.

128	64	32	16	8	4	2	1
1	0	0					

Now, continue this process until you have compared all the binary units. Remember to subtract the unit from the denary number if it is required. You should end with the following result:

128	64	32	16	8	4	2	1
1	0	0	1	0	1	1	0

CONTINUED

This means that if you convert the denary number 150 to 8-bit binary, you get the binary number 10010110.

A quick check that you can do on your calculation is to look at the last unit on the right of your binary number. If the denary number you are converting is an odd number, this number should be 1. If the denary number you are converting is an even number, this number should be 0.

Questions

4 Convert the denary number 58 to an 8-bit binary number.

5 Convert the denary number 123 to an 8-bit binary number.

If you need to convert a binary number into a denary number, you can use simple addition. You just need to add together all the units that are represented by the value 1.

Try converting the binary number 1010. A simple way to do this is to use a table. You can write the units above and the binary number below:

8	4	2	1
1	0	1	0

You just need to add together all the units that have a 1 below. This means the calculation is $8 + 2 = 10$. Therefore, if you are converting the binary number 1010 to denary, you get the denary number 10.

If you need to convert an 8-bit binary number, you need to extend the units so that you have 8 of them. Remember to increase by the power of 2 each time.

Try converting the binary number 10011001. You can write the values in a table again to make it clearer:

128	64	32	16	8	4	2	1
1	0	0	1	1	0	0	1

You just need to add together all the units again that have a 1 below. This means the calculation is $128 + 16 + 8 + 1 = 153$. Therefore, if you are converting the binary number 100011001 to denary, you get the denary number 153.

Questions

6 What is the largest denary number that can be created with an 8-bit binary number?

7 Convert the 8-bit binary number 01100011 to a denary number.

8 You need to be able to convert up to 16-bit binary values. Fill in the missing units in the table and calculate the denary number for the given binary number.

							256	128	64	32	16	8	4	2	1
0	1	1	0	0	0	1	0	1	0	0	0	1	0	0	1

REFLECTION

How did you know how to add in the correct missing units for Question 8? How did you work out each unit?

Hexadecimal

Hexadecimal is also a number system. Humans can use hexadecimal, but computers cannot process it. Like denary numbers, hexadecimal numbers also need to be converted to binary to be processed by a computer.

Hexadecimal is a base-16 system and uses 16 symbols, these are the numbers 0–9 and the letters A–F. This is to allow the denary values 0–15 to be represented in hexadecimal. This is shown in Table 1.1.

Denary	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hexadecimal	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Table 1.1: Hexadecimal representations of denary numbers

The reason for using the symbols A–F for the denary numbers 10–15 is so only one symbol is needed for the denary number in hexadecimal.

SKILLS FOCUS 1.2

CONVERTING BETWEEN DENARY NUMBERS AND HEXADECIMAL

To convert a denary number to hexadecimal, it is easier to convert it to binary first. For example:

First convert the denary number 7 to 4-bit binary.

8	4	2	1
0	1	1	1

To convert the binary number to hexadecimal you can use simple addition again. 4 + 2 + 1 = 7. If you look at the symbol table (Table 1.1), you can see that the hexadecimal for 7 is also 7. Therefore, the denary number 7 is 0111 as binary and 7 as hexadecimal.

So, what happens with denary numbers that are greater than 9?

First convert the denary number 12 to 4-bit binary.

8	4	2	1
1	1	0	0

To convert the binary number to hexadecimal you can use simple addition again. 8 + 4 = 12. If you look at the symbol table (Table 1.1), you can see that the hexadecimal for 12 is C. Therefore, the denary number 12 is 1100 as binary and 12 as hexadecimal.

Converting the denary number to binary first isn't necessary for denary numbers that are less than 16, but it makes the process easier when you need to convert larger denary numbers.

So, how do you convert a larger denary number to hexadecimal?

CONTINUED

First, convert the denary number 201 to 8-bit binary. You know you can use 8-bit binary because the denary number is less than 256.

128	64	32	16	8	4	2	1
1	1	0	0	1	0	0	1

Each hexadecimal symbol only uses 4 bits. Therefore, you need to split the 8-bit binary number into two 4-bit binary numbers.

8	4	2	1
1	1	0	0

8	4	2	1
1	0	0	1

You then convert each 4-bit binary number again using simple addition.

8 + 4 = 12

8 + 1 = 9

The hexadecimal for 12 is C and the hexadecimal for 9 is 9. Therefore, the denary number 201 converted to binary is 11001001 and to hexadecimal is C9.

To convert a hexadecimal number to a binary number of a denary number you can just reverse the process.

To convert the hexadecimal number 5E to a binary number you need to convert each symbol to 4-bit binary. Remember E is the number 14.

8	4	2	1
0	1	0	1

5

8	4	2	1
1	1	1	0

E

Therefore, the hexadecimal 5E converted to a binary number is 01011110.

To convert the hexadecimal 5E to denary, you can use the binary number that you have just calculated.

128	64	32	16	8	4	2	1
0	1	0	1	1	1	1	0

You can add all the units together to get the denary value. 64 + 16 + 8 + 4 + 2 =v94.

The hexadecimal 5E converted to a denary number is 94.

Questions

1 Convert the hexadecimal A2 to a binary number.
2 Convert the binary number 100111010001 to hexadecimal.
3 Convert the denary number 350 to hexadecimal.
4 Convert the hexadecimal 3AC to a denary number.

Why is hexadecimal used in computer science?

Programmers often use hexadecimal when writing and analysing computer programs. They do this because it is easy for a human to read hexadecimal, rather than binary.

For example, if a programmer looked at a set of data that is stored in registers, as binary it could look like this:

10011100101110111000111011100010111101011010

The programmer may be trying to find out if the data stored matches the correct data which should be:

10011100101110111001011011100010111101011010

At a glance, it is difficult to see whether the two values match or if an error has occurred.

If that stored data is represented in hexadecimal it would be:

9CBB8EE2F5A

Comparing it to the correct value that should be:

9CBB96E2F5A

It is much easier to see that the data does not match, and the programmer can identify that an error has occurred.

The benefits of using hexadecimal rather than binary for a programmer are:

- Hexadecimal is a shorter way of representing binary, so it takes up less space on the computer screen or piece of paper.
- Hexadecimal is easier for a programmer to read, write and understand.
- Hexadecimal is easier to debug and locate errors in data.

ACTIVITY 1.1

Each computer has a **MAC address**. This MAC address is written in hexadecimal. This is one of the ways that hexadecimal is used in computer science.

Use the internet to find three more uses of hexadecimal in computer science.

KEY WORD

MAC address: a unique address that is given to a computer at the manufacturing stage that can be used to identify the computer.

1.2 Binary manipulation and negative numbers

Binary addition

You need to be able to add together two 8-bit binary numbers. When binary numbers are added together there are four rules that you will need to use:

Rule 1 is 0 + 0 = 0

Rule 2 is 1 + 0 = 1

Rule 3 is 1 + 1 = 10 (this is because 10 in binary is 2)

Rule 4 is 1 + 1 + 1 = 11 (this is because 11 in binary is 3)

You can apply these rules to add the binary numbers 10010100 and 00011110.

	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0

You need to start at the right and begin applying the rules. The first calculation is 0 + 0. Rule

The next calculation is 0 + 1. Rule 2 means the result is 1.

	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
							1	**0**

The next calculation is 1 + 1. Rule 3 means the result is 10. You need to write the 0 in the answer section and carry the 1 to the next calculation.

					1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
						0	**1**	**0**

The next calculation is 1 + 0 + 1, which results in 1 + 1. Rule 3 means the result is 10. Write the 0 in the answer section again and carry the 1.

				1	1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
					0	**0**	**1**	**0**

The next calculation is 1 + 1 + 1. Rule 4 means the result is 11. Write 1 in the answer section and carry 1.

			1	1	1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
				1	**0**	**0**	**1**	**0**

The next calculation is 1 + 0 + 0, which results in 1 + 0. Rule 2 means the result is 1. Write 1 in the answer section. There is no carry this time.

			1	1	1			
	1	0	0	1	0	1	0	0
+	0	0	0	1	1	1	1	0
			1	**1**	**0**	**0**	**1**	**0**

The next calculation is 0 + 0. Rule 1 means the result is 0. Write 0 in the answer section.

			1	1	1			
+	1	0	0	1	0	1	0	0
	0	0	0	1	1	1	1	0
		0	**1**	**1**	**0**	**0**	**1**	**0**

The final calculation is 1 + 0. Rules 2 means the result is 1. Write 1 in the answer section.

			1	1	1			
+	1	0	0	1	0	1	0	0
	0	0	0	1	1	1	1	0
	1	**0**	**1**	**1**	**0**	**0**	**1**	**0**

Therefore, when the binary numbers 10010100 and 00011110 are added, the answer is 10110010.

Questions

3 Add the 8-bit binary values 00011001 and 01000011.

4 Add the 8-bit binary values 00100110 and 01010111.

You should now know that the largest number that can be stored in an 8-bit binary register is 255. What happens when two 8-bit binary values are added, and the result is larger than 255?

The following example, which is almost complete, is very much like the previous calculation with a slightly different binary number. Can you spot the difference?

			1	1	1			
+	1	0	0	1	0	1	0	0
	1	0	0	1	1	1	1	0
		0	**1**	**1**	**0**	**0**	**1**	**0**

If you now try and do the final calculation, that would be 1 + 1 = 10. You write 0 in the answer section, but there is nowhere to carry the 1. This is because the two 8-binary binary values are added the result is 306, which is greater than 255. Therefore, the number cannot fit in the 8-bit register and an **overflow error** is created.

			1	1	1			
+	1	0	0	1	0	1	0	0
	1	0	0	1	1	1	1	0
(1)	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**0**

Overflow error

Each computer has a predefined limit, for example, 16-bit registers. What is the largest number that could be stored in a 16-bit register? If a number larger than this is generated in a process, an overflow error will occur.

KEY WORD

overflow error: a type of error that occurs when a number larger than a register can store is generated.

Logical binary shifts

Another process than can occur with binary numbers that are stored in a register is called a logical binary shift. This kind of shift can move the binary values in the number to the left or to the right.

Consider the binary number:

0	0	1	0	1	1	0	0

If a logical binary shift to the left is performed on the binary number, each value in the binary number is moved one place to the left. The result would be:

0	1	0	1	1	0	0	

You should notice that the left most 0 has been removed from the register. This is because there was no place to the left to shift this value. You should also notice that there is now an empty section in the register at the right. When a binary number is shifted, the resulting empty sections are filled with a 0.

0	1	0	1	1	0	0	0

If you convert the first binary number to denary, you get the denary number 44. If you convert the binary number after the logical left shift has been performed, you get the denary number 88. This means that each logical shift to the left multiples the binary number by 2.

Consider the binary number again:

0	0	1	0	1	1	0	0

If a logical shift to the right is performed on the binary number, each value in the binary number is moved one place to the right. The result would be:

	0	0	1	0	1	1	0

You should notice that the right most 0 has been removed from the register. This is because there was no place to the right to shift this value. You should also notice that there is now an empty section in the register at the left. When a binary number is shifted, the resulting empty sections are, again, filled with a 0.

0	0	0	1	0	1	1	0

You already know that the first binary number converted to a denary number is 44. If you convert the binary number after the logical right shift has been performed, you get the denary number 22. This means that each logical shift to the right divides the binary number by 2.

Consider another binary number:

1	1	1	0	1	1	0	0

When a logical left shift is performed on the binary value the result is:

1	1	0	1	1	0	0	0

When the first binary number is converted to a denary number, the result is 236. When the binary number is converted after the logical shift to the left has been performed, the denary value is 216. You should notice that 236 multiplied by 2 is not 216. Therefore, one of the problems with a logical shift is that data can lose its integrity as vital bits can be removed from the register.

More than one logical shift can be performed on a register. For example, two logical shifts to the left can be performed. This time, each value in the register is shifted two places to the left.

The binary value:

1	1	1	0	1	1	0	0

would become:

1	0	1	1	0	0	0	0

Questions

5 Describe what happens in a logical binary shift.

6 Perform **two** logical shifts to the right on the binary number 11101100.

Negative binary numbers

All the binary numbers that you have used so far have been positive binary numbers. Numbers can sometimes be negatives number though, for example, −150. How are negative numbers represented as binary? Binary only has two values that can be used, a 0 and a 1, so you can't just put a minus symbol (−) in front of the number, as that symbol doesn't exist in binary. Therefore, a method exists that can be used to represent negative binary numbers and it is called two's complement. Most modern computers use this method.

The method of two's complement is a simple one. First you invert all the values in the binary number (change the 1s to 0s and the 0s to 1s), then you add 1 to the result. You can use this process to find out how the denary value −35 is represented as binary.

Convert 35 to a binary number:

0	0	1	0	0	0	1	1

Invert each of the values in the binary number:

1	1	0	1	1	1	0	0

Then simply add 1:

1	1	0	1	1	1	0	1

11011101 is the binary representation of −35 using the two's complement method.

1.3 How do computers represent text, images and sound?

You have already learnt that computers only process binary. This means that all data must be converted to binary to be processed by a computer. You have already looked at converting numbers, but text, images and sound all have to be converted to binary too, so they can be processed by a computer. How is this done?

Converting text to binary

When a computer converts text to binary to be processed it uses a tool called a **character set**. There are two main types of character set that a computer can use, ASCII and Unicode. An ASCII character set represents characters using 8-bit binary numbers. This means that it can represent up to 256 characters (0 to 255). This is fine for certain languages, but some languages have an alphabet that requires more than 256 characters; therefore, Unicode was invented. A Unicode character set represents characters using 16-bit binary numbers. This means it can represent a much greater number of characters, approximately 65 000. This means that many more characters can be developed in different alphabets and also symbols such as emojis can also be used.

Each character has a set binary value, for example, the letter A is 01000001.

KEY WORD

character set: it contains all the characters that are in that character set and the binary value that is assigned to each character.

ACTIVITY 1.2

Use the internet to find an example of an ASCII character set and a Unicode character set. What kind of different characters are represented? Are upper and lowercase letters the same?

Using the ASCII table, try to decode this message:

01000011011011110100110101110000011101010111010001100101011100100101001101

10001101101001010001010110111001100011011001010110100101110011011001100111

01010110111000100001

Remember, each letter will be represented as an 8-bit binary number.

Try writing a message for your friend to decode.

Converting images to binary

An image is made up of tiny dots called **pixels**. If an image was simply created using the colours black and white, each pixel would be either black or white. The binary value 1 could be used to represent the colour black and the binary value 0 could be used to represent the colour white.

KEY WORD

pixel: a very small dot of colour that is displayed with many others to create an image.

If each pixel is converted to its binary value, a data set such as the following could be created:

0011111000100000101000000011001010011000000011010001011001110010100000100
01111100

If the computer is informed that the image that should be created using this data is 9 pixels wide and 10 pixels high, it can set each pixel to black or white and create the image (See Figure 1.2). The type of data that is used to provide information, such as the dimensions and **resolution** of an image, is called **metadata**.

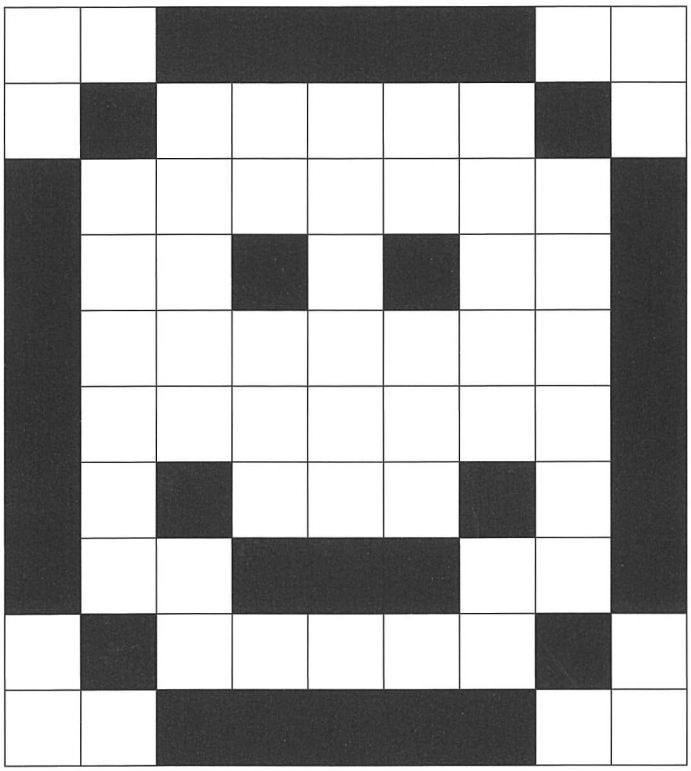


Figure 1.2: An image created just from black and white pixels

Most images use a lot more colours than black and white. Each colour has its own binary values. Colours are created by computer screens using the Red Green Blue (RGB) colour system. This system mixes the colours red, green and blue in different amounts to achieve each colour.

ACTIVITY 1.3

Use the internet to find an RGB colour scale. You should notice that each colour in the scale is first presented as hexadecimal. The first two hexadecimal values are the amount of red, the second two the amount of blue and the last two the amount of green.

Practise your hexadecimal to binary conversion by converting your favourite colour to its binary value.

KEY WORDS

resolution: the dimensions of an image.

metadata: additional data that is stored with an image that can provide information such as the dimensions of the image and the time and date the image was taken.

Most images are created using thousands of pixels. Each of those pixels needs quite a lot of data. Multiply that by the thousands of pixels that make up the image and that is a huge amount of data. This is why image files can often be so large in size.

Each image has a resolution and a **colour depth**. The resolution is the number of pixels wide and the number of pixels high that are used to create the image. The colour depth is the number of bits that are used to represent each colour. For example, each colour could be represented using 8-bit, 16-bit or 32-bit binary numbers (see Figure 1.3). The greater the number of bits, the greater the range of colours that can be represented.

KEY WORD

colour depth: the number of bits that are used to create each colour in an image

If the colour depth of an image is reduced, the quality of the image is often reduced.



Figure 1.3: 8-bit colour 16-bit colour 32-bit colour

Can you see how the quality of the image is reduced in comparison to the number of bits used?

If the resolution or the colour depth of an image is changed, this will have an effect on the size of the image file. If the resolution is increased, the image will be created using more pixels, so more data will need to be stored. If the colour depth of the image is increased, each pixel will need more data to display a greater range of colours so more data will need to be stored. Both will result in a larger file size for the image.

COMPUTER SCIENCE IN CONTEXT

People often have to sacrifice the quality of their images when displaying them online. This is because having files that are very large in size can take more time to download. This may frustrate users as it may mean they have to wait for a website to load, due to the number of large files it contains.

Converting sound to binary

Sound is made up of sound waves. When sound is recorded, this is done at set time intervals. This process is known as **sound sampling**.

KEY WORD

sound sampling: a little piece of sound that is recorded at regular time intervals.

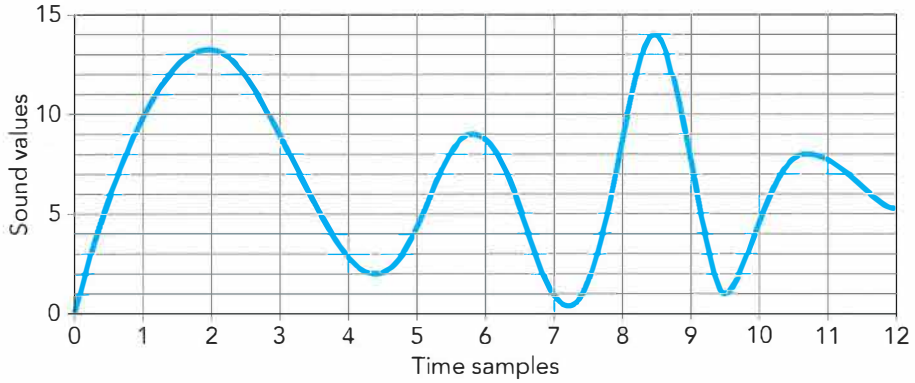


Figure 1.4: A sound wave

Figure 1.4 is an example of a sound wave.

If you recorded the sound at each time sample, you would have the following set of values:

Time sample	1	2	3	4	5	6	7	8	9	10	11	12
Sound value	9	13	9	3.5	4	9	1.5	9	8	5	8	5.5

Each sound value is converted to binary to be processed by the computer. Each sound value is then used to playback the sound, when the recording is played. If the recorded sound wave is played it would now look like Figure 1.5:

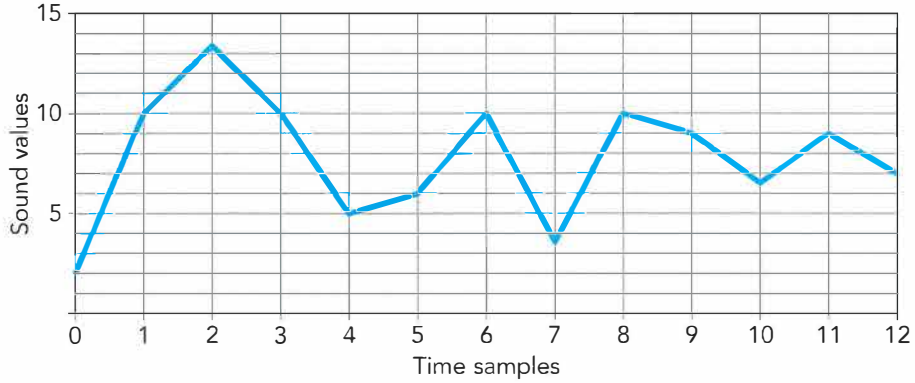


Figure 1.5: A sound wave created by playing back a recorded sound

You will notice that the two sound waves look different. This is because the number of samples taken is too far apart, so vital data in between time samples has been lost. This will result in a lack of accuracy in the sound that is recorded. Therefore, more samples need to be taken each second to improve the accuracy of the recorded sound. The number of samples taken in a second is called the **sample rate**. Sample rates are measured in hertz. 1 hertz is equal to 1 sample per second. A common sample rate is 44.1 khz (kilohertz), this would require 44 100 samples to be taken each second. That is a lot of data! If the sample rate is increased, the amount of data required for the recording is increased. This increases the size of the file that stores the sound.

Further data is required to represent sound, and this is the **sample resolution**. This is the number of bits that are used to represent each sample. A common sample resolution is

KEY WORDS

sample rate: the number of samples recorded each second.

sample resolution: the number of bits that are used to record each sound sample.

16-bit. The higher the sample resolution, the greater the variations in amplitude that can be stored for each sample. This means that aspects such as the loudness of the sound can be recorded more accurately. This will also increase the amount of data that needs to be stored for each sample. This increases the size of the file that stores the sound.

1.4 Measuring data storage

Earlier in the chapter, you learnt that all data need to be converted to digital data to be processed by a computer, and you now know how that is done for different types of data. But how is data measured when it is stored?

COMPUTER SCIENCE IN CONTEXT

There are times when you will need to know the size of a data file, such as when you are attaching a file to an email. Most email providers have a file size limit, so you need to know the size of that file, to know if it can be attached to your email.

Data is measured using different units, which are shown in Table 1.2.

Unit	Description
Bit	This is the smallest measurement for data
Nibble	There are 4 bits in a nibble
Byte	There are 8 bits in a byte
Kibibyte (KiB)	There are 1024 bytes in a Kibibyte
Mebibyte (MiB)	There are 1024 Kibibytes in a Mebibyte
Gibibyte (GiB)	There are 1024 Mebibytes in a Gibibyte
Tebibyte (TiB)	There are 1024 Gibibyte in a Tebibyte
Pebibyte (PiB)	There are 1024 Tebibytes in a Pebibyte
Exbibyte (EiB)	There are 1024 Pebibytes in an Exbibyte

Table 1.2: Data measuring units

Although most computers will calculate the size of a file for you, you need to be able to show that you can do this manually for the exam. This is just for an image file and a sound file.

SKILLS FOCUS 1.3

CALCULATING THE SIZE OF A FILE

There are several pieces of information that you need to know to calculate the size of an image file, these are:

- the number of images in the file
- the resolution of the image
- the colour depth of the image.

CONTINUED

When you know this information, you can perform a simple calculation to get the size of the image file, which is:

width of image × height of image × colour depth of image × number of images in file

Therefore, if you have the following information about an image file:

- 10 images in the file
- image resolution is 100 × 150
- colour depth is 8-bit colour,

you can calculate the size of the image file as follows:

- 100 × 150 = 15,000 (this means there are 15 000 pixels in the image)
- 15 000 × 8 = 120,000 (this means that there are 120 000 bits in each image)
- 120 000 × 10 = 1 200 000 (this means there are 1 200 000 bits in the image file).

You will be asked to calculate the size of an image file to a certain unit of measurement. The answer given here (1 200 000) is the number of bits that are in the image file. If you wanted to know how many bytes are in the image file, you need to divide the final answer by 8, as there are 8 bits in each byte, for example:

1 200 000 ÷ 8 = 150 000 bytes

If you wanted to know how many kibibytes the file was, you need to divide the answer in bytes by 1024, for example:

150 000 ÷ 1024 = 146.5 KiB (to 1 decimal place)

Questions

1 How many times would you need to divide the answer in bytes by 1024 to find the answer in Gibibytes?

2 How many pixels would there be in an image that has a resolution of 1000 × 1500?

There are several pieces of information that you need to know to calculate the size of a sound file, these are:

- the length of the soundtrack
- the sample rate of the soundtrack
- the sample resolution of the soundtrack.

When you know this information, you can perform a simple calculation to get the size of the sound file, which is:

sample rate × sample resolution × length of soundtrack

Therefore, if you have the following information about a sound file:

- length of 30 seconds
- sample rate of 44,100 hz
- sample resolution of 8 bits,

you can calculate the size of the sound file as follows:

- 44 100 × 8 = 352 800 (this means there are 352 800 bits per second)
- 352 800 × 30 = 10 584 000 (this means there are 10 584 000 bits in the sound file).

CONTINUED

You will be asked to calculate the size of a sound file to a certain unit of measurement. The answer given (10 584 000) is the number of bits that are in the sound file. If you wanted to know how many bytes are in the sound file, you need to divide the final answer again by 8, as there are 8 bits in each byte, for example:

$$10\ 584\ 000 \div 8 = 1\ 323\ 000 \text{ bytes}$$

If you wanted to know how many mebibytes the file was, you need to divide the answer in bytes by 1024, then by 1024 again, for example:

$$1\ 323\ 000 \div 1024 \div 1024 = 1.3 \text{ MiB (to 1 decimal place)}$$

1.5 Data compression

You may already have experience of having large files and struggling to have enough storage space to store them. One thing that can be done to reduce the size of a file is **compression**.

Compressing a file will reduce the size of the file that can have several benefits, such as:

- Not as much storage space is needed to store the file.
- It will take less time to transmit the file from one device to another.
- It will be quicker to upload and download the file.
- Not as much bandwidth is needed to transmit the file over the internet.

There two type of compression that can be used, **lossy** and **lossless**. Each compresses the data in a different way.

Lossy compression uses a compression algorithm that finds the unnecessary and redundant data in the file. This data is permanently removed from the file. This type of compression is mainly used on an image file or a sound file.

Unnecessary data in an image file that could be removed are colours that the human eye cannot distinguish. This means that a lossy compression algorithm for an image file could reduce the size of the file by reducing the colour depth. It could also reduce the size of the file by reducing the image resolution, reducing the number of pixels used to create the image.

Unnecessary data in a sound file that could be removed are sounds that cannot be heard by the human ear. It could also be the removal of softer sounds that are played at the same time as louder sounds, as humans will only hear the louder sound. The method that is used to find these kinds of sounds in a sound file is called **perceptual music shaping**. This is the most common type of method that is used in the file format MP3. The size of a sound file can also be reduced by reducing the sample rate and the sample resolution.

KEY WORDS

compression: a method that uses an algorithm to reduce the size of a file.

lossy: a compression method that reduces the size of a file by permanently removing data.

lossless: a compression method that reduces the size of a file by temporarily altering the data.

KEY WORD

perceptual music shaping: a process that is used in lossy compression that removes sounds that are not normally heard by the human ear.

Questions

7 Why do you think lossy compression is not commonly used on text files?

8 Will lossy compression affect the quality of an image? Why?

9 Will lossy compression affect the quality of a sound recording? Why?

SELF ASSESSMENT

What were the points that you thought about regarding lossy compression to answer question 7? Did you relate these points back to why lossy compression is not suitable for a text file, or did you just make points about the drawbacks of lossy compression in general?

Lossless compression uses a compression algorithm to reduce the size of the file, without permanently removing any data. A common method that can be used to do this is by finding repeating data in the file and grouping this data together. For example, consider the following message:

WHEN IT IS SNOWING HEAVILY LOOK OUTSIDE.
LOOK OUTSIDE IT IS SNOWING HEAVILY.

Excluding the spaces between the words and the full stops, the message has a total of 62 characters. 1 character requires 1 byte of storage, so 62 bytes of storage would be needed to store this message.

When you look at the message, you should notice that it consists of words that are mostly repeated. Instead of storing all 62 characters individually, the words and the positions at which they occur in the message can be stored in a lookup table, for example:

Word	Position(s) in the message
WHEN	1
IT	2, 10
IS	3, 11
SNOWING	4, 12
HEAVILY	5, 13
LOOK	6, 8
OUTSIDE	7, 9

The lookup table needs 1 byte for each character in each word and 1 byte for each position the word occurs in the message. Therefore, 33 bytes are needed to store the words and 13 bytes to store the positions, giving a total of 46 bytes. This is much less than the 62 bytes we required with our original method. No data has been lost and we have reduced our storage requirements by 26%, quite a saving! To recreate the message, the computer simply retrieves the words and places them in the positions allocated.

A similar process can be applied to an image file with a lossless compression algorithm called **run length encoding (RLE)**. This algorithm will group together repeating pixels and store how many times they occur. For example, consider the simple image in Figure 1.6:

KEY WORD

run length encoding (RLE): An algorithm that groups together repeating patterns and indexes them.

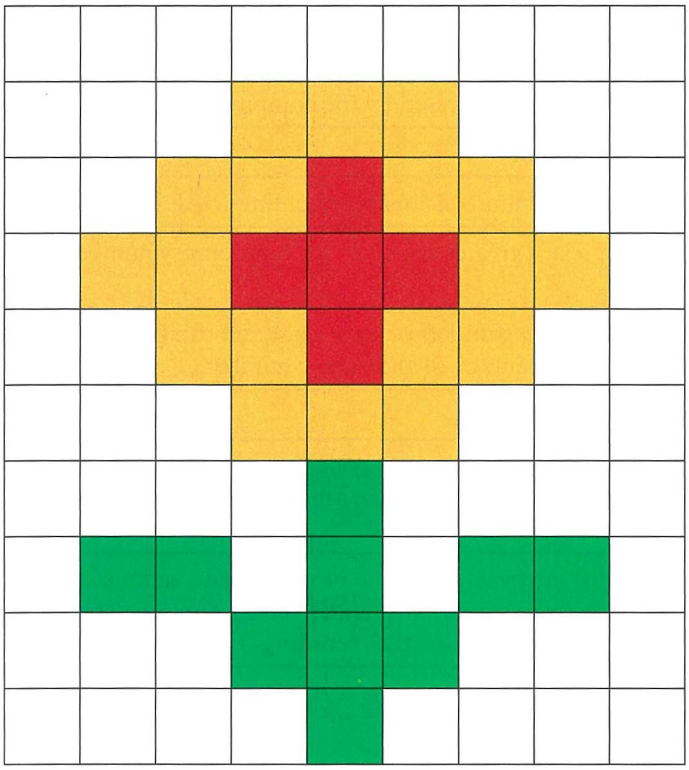


Figure 1.6: A simple pixel image

If W is white, Y is yellow, R is red and G is green, RLE could compress the image into the following data:

12W, 3Y, 5W, 2Y, 1R, 2Y, 3W, 2Y, 3R, 2Y, 3W, 2Y, 1R, 2Y, 5W, 3Y, 7W, 1G, 5W, 2G, 1W, 1G, 1W, 2G, 4W, 3G, 7W, 1G, 4W.

This means that the amount of data can be reduced as pixels are grouped together and the data for each pixel does not need to be stored. The resolution of the image is stored with the data, so that the computer knows the size to recreate it when the data is decompressed.

ACTIVITY 1.4

Why don't you try creating a simple image and writing out the RLE data for it? You could then give it to your friend, along with the image resolution, and they can have fun trying to recreate your image.

Peer Assessment

Ask your friend whether you got the RLE correct for your image in Activity 1.4. Ask them if you could have done anything to make your RLE any clearer. Ask them if you missed any important data.

SUMMARY

Computers require any data to be converted to binary before it can be processed by the computer.
Different number systems exist such as denary, binary and hexadecimal.
Hexadecimal is used in computer science for uses such as aiding programmers and representing MAC addresses.
Binary numbers can be added using four rules. If this creates a result greater than 255 for 8-bit binary numbers, an overflow error occurs.
Logical binary shifts can be performed on binary numbers. These can shift left or shift right and can be done multiple times. This changes the binary number and can affect the accuracy of the binary number.
Text is converted to binary using a character set such as ASCII or Unicode.
Images are converted to binary by taking each pixel and providing it with a colour code that is converted to a binary value. The larger the resolution and colour depth of an image, the greater the amount of data that will need to be stored in the image file. This also increases the quality of the image.
Sound is converted to binary by sampling the sound at a set time interval. The value of each sound sample is converted to binary. The greater the sample rate and resolution of the sound, the greater the amount of data that will need to be stored in the sound file. This also increases the accuracy of the recording.
Data is measured in different units such as bits, bytes and mebibytes.
Data can be compressed, using lossy or lossless compression, to reduce the size of a file. This means it will take up less storage space and it can be quicker to transmit the file.

EXAM-STYLE QUESTIONS

1 There are two different types of data, known as analogue data and digital data.

a **State** what is meant by analogue data. [1]

b State what is meant by digital data. [1]

[Total: 2]

2 **Describe** the difference between a denary number system and a binary number system. [4]

3 A music venue has a small display screen built into the top of each seat. The display screen displays the number of the seat. Each seat number is displayed as hexadecimal.

a Convert the two given seat numbers into 12-bit binary values. [4]

- 21A
- 90D

b Write the seat number that would be displayed for the binary number 000010011100. [2]

[Total: 6]

4 Two registers store the given 8-bit binary values. Register 1 stores 00110111 and register 2 stores 01110011.

a Add the two binary values and write the result of the calculation. Show all your working. [3]

COMMAND WORDS

state: express in clear terms.

describe: state the points of a topic / give characteristics and main features.

CONTINUED

b A logical left shift of three places is performed on Register 1. Write the contents of register 1 after the logical shift has been performed. [1]

[Total: 4]

5 **Show** how the value −28 is represented in binary using two's complement. Show all your working. [2]

6 A student wants to store some images that they have taken with their digital camera. They do not have enough storage space on their external hard disk drive to store the images at their current file size.

a State how the student can reduce the amount of data stored for each image. [1]

b The student has 20 images to store, that have a resolution of 150 × 250 and a colour depth of 16-bit.

Calculate the size of the file that will store the images in MiB. [3]

[Total: 4]

7 A blogger displays images of their daily life on their blog. They also have sound files of hints and tips that they record available on their blog.

a The blogger compresses the images that they display on their blog.

Give *two* benefits of compressing an image file. [2]

b The blogger uses lossless compression to compress the sound file on their blog.

Explain how lossless compression will compress the sound file that the blogger makes available on their blog. [4]

[Total: 6]

8 Four file sizes are given.

Tick (✓) to show which file size is the largest.

File size	Tick (✓) the largest file size
1 GiB	
2033 bytes	
1050 MiB	
1500 KiB	

[1]

COMMAND WORDS

show (that): provide structured evidence that leads to a given result.

calculate: work out from given facts, figures or information.

give: produce an answer from a given source or recall / memory.

explain: set out purposes or reasons / make the relationships between things evident / provide why and/or how and support with relevant evidence.

SELF-EVALUATION CHECKLIST

After studying this chapter, think about how confident you are with the different topics. This will help you to see any gaps in your knowledge and help you to learn more effectively.

You might find it helpful to rate how confident you are for each of these statements when you are revising. You should revisit any topics that you rated 'Needs more work' or 'Getting there'.

CONTINUED

I can...	See topic	Needs more work	Getting there	Confident to move on
explain why data needs to be converted to binary to be processed by a computer.	1.1			
convert between denary numbers and binary numbers.	1.1			
convert between denary numbers and hexadecimal.	1.1			
convert between binary numbers and hexadecimal.	1.1			
provide examples of the use of hexadecimal in computer science.	1.1			
explain why hexadecimal is used in computer science.	1.1			
add two 8-bit binary numbers.	1.2			
explain what is meant by an overflow error.	1.2			
perform a logical binary shift on a binary number, including left and right shifts and multiple shifts.	1.2			
use two's complement to represent negative numbers.	1.2			
explain how character sets are used by a computer to represent text.	1.3			
explain how pixels are used by a computer to create images.	1.3			
explain how the file size of an image is affected by the image resolution and colour depth.	1.3			
explain how a sound wave is sampled by a computer.	1.3			
explain how the file size of a sound recording is affected by the sample rate and sample resolution.	1.3			
understand how the size of a data file is measured.	1.4			
calculate the size of an image file.	1.4			
calculate the size of a sound file.	1.4			
understand the need for data compression and the effect it has on a file.	1.5			
explain how different types of data are compressed using lossy compression.	1.5			
explain how different types of file are compressed using lossless compression.	1.5			